

KROZENDA

MULTI-VENDOR B2B & B2C MARKETPLACE

Payment & Settlement Flow

How money moves through Krozenda — from the customer's checkout, through the multi-vendor split, to automated vendor settlement and automated payroll.

Covers Razorpay Route for marketplace settlement, the RazorpayX Payouts API for automated salary and platform disbursements, refunds, RTO reversal, reconciliation, and the RBI compliance boundary between the two.

Document	Payment & Settlement Flow — technical and commercial explainer
Audience	Client stakeholders, finance team, and the implementation team
Platform	Krozenda — Node.js / Express / MongoDB · React · React Native
Payment partner	Razorpay — Payment Gateway, Route, and RazorpayX Payouts
Version	1.0 (Draft for client review)
Date	[DD Month YYYY]
Prepared by	Appzeto

Read section 2 first. Razorpay Route and the Razorpay Payout API are two different systems that move money in two different ways, under two different regulatory regimes. Which one we use for which purpose is the single most important decision in this document, and getting it wrong carries regulatory exposure — not just technical rework.

§ Contents

1. Executive summary	11. Automated payroll via RazorpayX Payouts
2. The two money rails	12. The Payout API in detail
3. Why Route is mandatory — the RBI position	13. Payout automation and scheduling
4. Flow 1 — Customer checkout and payment capture	14. Webhooks and reconciliation
5. Flow 2 — Order split and commission snapshot	15. Failure handling and edge cases
6. Flow 3 — Vendor onboarding as a linked account	16. Security and key management
7. Flow 4 — Automated vendor settlement	17. Tax treatment — TCS and TDS
8. Worked example with real numbers	18. What the client must provide
9. Flow 5 — Refunds and partial refunds	19. Open decisions
10. Flow 6 — RTO and settlement reversal	20. Implementation sequence

1. Executive summary

Krozenda collects money from customers online and distributes it to three kinds of recipient: independent marketplace sellers, dropshipping partners, and Krozenda itself for its own stock and its commission. On top of that, the platform must pay its own staff. This document explains, end to end, how each of those movements works.

The design rests on four decisions:

Decision	What it means in practice
Online payment only	UPI, credit card, debit card, net banking and wallets. No Cash on Delivery. Every order is fully paid before it is processed, so there is no collection risk and no cash reconciliation.
Vendor money moves on Razorpay Route	The customer's payment is split by Razorpay at the moment of capture. Krozenda's commission and each vendor's share are routed separately. Krozenda never pools and re-disburses vendor funds.
Vendor payout is automatic, not manual	Each vendor's share is transferred on hold at order time and released automatically once the order is delivered and the return window closes. No one presses a button per order.
Payroll moves on RazorpayX Payouts	Staff salaries and platform-owned disbursements are paid from Krozenda's own account through the Payout API, on a schedule. This is Krozenda's own money, so this rail is appropriate here.

The short version. Vendor money never sits in Krozenda's bank account — Razorpay splits it at source and holds the vendor's share until the order is safely delivered. Krozenda's own money — commission earned, own-stock revenue — settles to Krozenda's account normally, and salaries are paid out of it automatically each cycle.

2. The two money rails

Razorpay sells several products. Two of them move money out to other people, and they are not interchangeable. Understanding the difference is the foundation of everything below.

Rail A — Razorpay Route

Used for: marketplace vendor settlement.

The customer pays once. Razorpay splits that single payment and routes each share to a separate *linked account* — one per vendor — before the money ever reaches Krozenda's bank account.

Krozenda is a marketplace facilitator here. The money passing to vendors is **not Krozenda's money** and never becomes Krozenda's money.

Rail B — RazorpayX Payouts

Used for: payroll and platform disbursements.

Krozenda holds a balance in its own RazorpayX current account and instructs a transfer to a named beneficiary by IMPS, NEFT, RTGS or UPI.

This is Krozenda paying out **its own money** — salaries, reimbursements, incentives — exactly as a business pays any supplier or employee.

2.1 Which rail for which purpose

Money movement	Rail	Why
Marketplace seller's share of a sale	Route	Third-party money. Must be split at source.
Dropshipping partner's share of a sale	Route	Same as above.
Krozenda's commission	Route (retained)	Retained in the split; settles to Krozenda's account on the normal gateway cycle.
Own Stock sale proceeds	Route (retained)	Krozenda is the seller; the full amount is Krozenda's.
Customer refund	Refunds API	Reverses the original payment to the original instrument. Not a payout.
Staff salary	Payouts	Krozenda's own money to its own employees.
Employee reimbursement	Payouts	Same.
Vendor incentive or marketing rebate paid by Krozenda	Payouts	Krozenda's own money, paid as a business expense — distinct from the vendor's sale proceeds.
Goodwill compensation to a customer outside the refund window	Payouts	Cannot be reversed through the gateway once the window has closed.

The line that must not be crossed. Vendor sale proceeds must never travel on Rail B. Collecting customers' payments into Krozenda's own account and then paying vendors out of it by Payout API is *aggregation*, and under the RBI Payment Aggregator framework that activity requires a Payment Aggregator licence, which Krozenda does not hold. It would also mean vendor money sitting in Krozenda's account, exposed to Krozenda's creditors. This is a licensing question, not an engineering preference — see section 3.

3. Why Route is mandatory — the RBI position

Under the RBI's *Guidelines on Regulation of Payment Aggregators and Payment Gateways*, an entity that collects funds from customers on behalf of merchants and then settles those funds to the merchants is acting as a Payment Aggregator, and must be authorised to do so. Funds so collected must sit in an escrow account with a scheduled commercial bank, not in the entity's own operating account.

Krozenda is not an authorised Payment Aggregator. The compliant path is therefore to use one that is — Razorpay — and to have Razorpay perform the split itself:

- Razorpay holds the funds in its own regulated escrow.
- Razorpay performs the split into linked accounts.
- Razorpay settles each linked account to its own registered bank account on its own cycle.
- Krozenda instructs the split, but never custodies the vendor's share.

Each vendor is onboarded as a Route **linked account** with its own KYC held by Razorpay. A vendor that has not completed that onboarding cannot receive money, and the platform must block it from selling until it has — see

4. Flow 1 — Customer checkout and payment capture

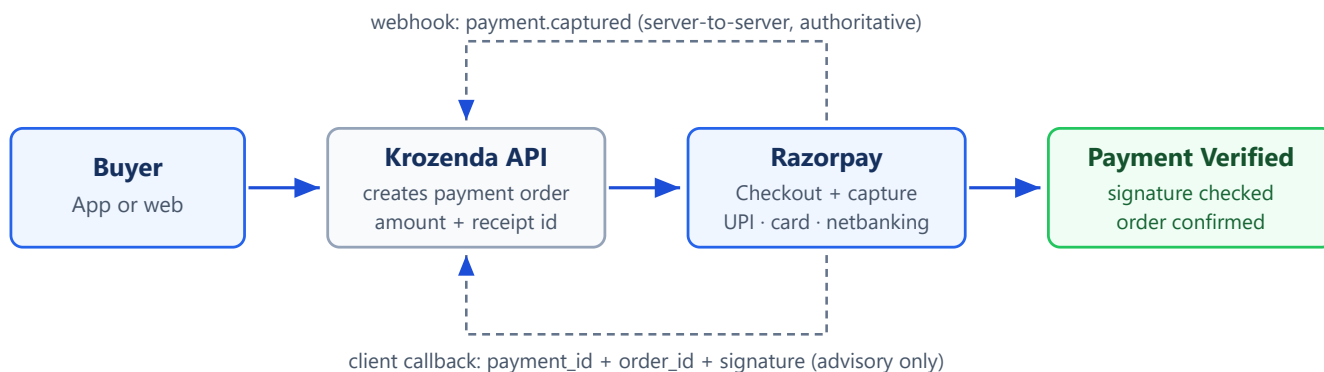


Figure 1 — Checkout. The webhook is the source of truth; the browser callback is never trusted on its own.

4.1 Step by step

#	Step	What happens
1	Cart validated	Server re-prices the cart from the database — never from what the client sent. Stock, MOQ, B2B tier eligibility, coupon validity and PIN-code serviceability are all re-checked at this moment.
2	Payment order created	Krozenda's server calls Razorpay to create a payment order for the final total, with our internal order reference as the receipt. The amount is fixed server-side, in paise.
3	Checkout opened	The buyer completes payment on Razorpay's interface. Card numbers, CVV and UPI PINs are entered there and never touch Krozenda's servers.
4	Callback received	The client returns a payment id, order id and signature. We verify the signature, but treat this only as a hint that payment may have succeeded.
5	Webhook received	Razorpay calls our server directly with <code>payment.captured</code> . We verify the HMAC signature against our webhook secret. This is the event that confirms the order.
6	Order confirmed	The order moves to <i>Payment Verified</i> , sub-orders are created, and vendors are notified. The buyer sees confirmation.

Why the webhook and not the callback. The browser callback can be lost — the app is killed, the network drops, the user closes the tab mid-redirect. It can also be forged by a determined attacker replaying a response. The server-to-server webhook is neither. We build so that a customer whose app crashed the instant after paying still gets a confirmed order, because the webhook arrives regardless.

5. Flow 2 — Order split and commission snapshot

A single cart can contain items from several sellers. At confirmation, the parent Order is split into one **Sub-Order per seller**, and each sub-order carries its own financial identity from that moment on.

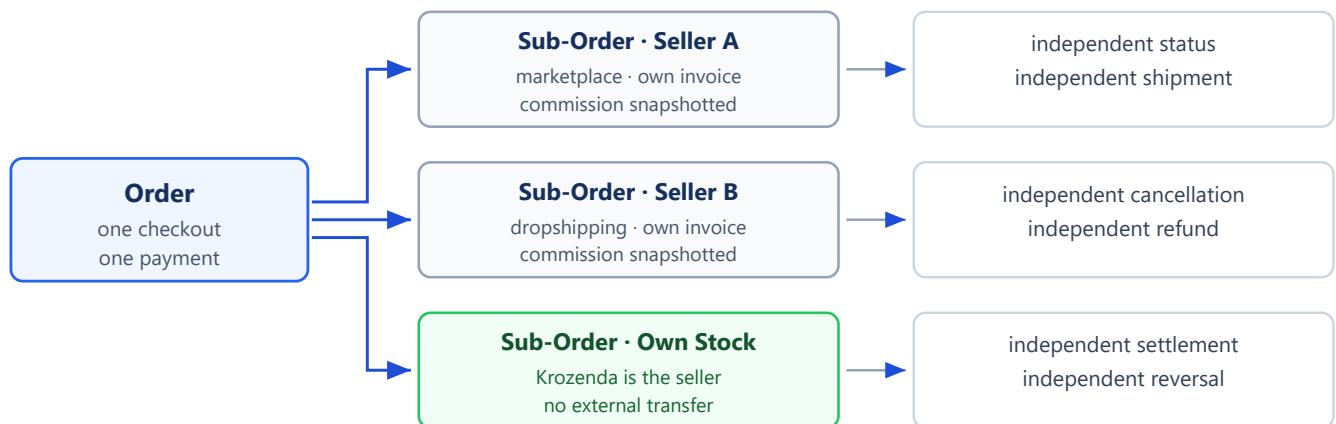


Figure 2 — One payment, many sub-orders. Each sub-order is financially self-contained.

5.1 Commission is snapshotted, never recomputed

Commission is resolved at the moment the order is placed, using a most-specific-wins lookup, and the resulting figure is written onto the sub-order:

product-specific → seller-specific → category-specific → company-specific → default rate

Once written, that number never changes. If the client later raises the default commission from 8% to 10%, orders placed yesterday still settle at 8%. This matters commercially — it is what makes historical settlement reports reproducible and defensible in a vendor dispute — and it is why the commission figure lives on the sub-order record rather than being looked up from live configuration at settlement time.

6. Flow 3 — Vendor onboarding as a linked account

A vendor cannot receive a single rupee until Razorpay has onboarded and verified it. The sequence is:

1. **Vendor applies on Krozenda** and uploads its documents — PAN, GST certificate, cancelled cheque, address proof, and category licences such as FSSAI.
2. **Krozenda's admin reviews them manually** and approves or rejects with a reason. This is a human check against the documents supplied; it is not an automated verification against a government database.
3. **Krozenda creates a Route linked account** for the approved vendor, submitting the vendor's legal name, business type, contact details, and bank account.
4. **Razorpay runs its own KYC** on the linked account. This is independent of Krozenda's review and can take several working days. Razorpay may request further documents directly.
5. **The storefront activates** only once the linked account reaches an active, transfer-eligible state.

Operational consequence to plan for. Razorpay's KYC on a linked account is not instant and is outside Krozenda's control. A vendor approved by your admin on Monday may not be able to receive money until the following week. The Seller Panel must show this state honestly — "approved by Krozenda, awaiting payment partner verification" —

or your support team will field the complaints. The platform must also hard-block order assignment to any vendor whose linked account is not yet transfer-eligible, otherwise the money has nowhere to go at settlement.

Gap flagged: no penny-drop verification. Bank account verification by penny drop is outside the agreed scope. Without it, a vendor who mistypes an account number will have a linked account that looks valid but fails at transfer time, or — worse — succeeds into someone else's account. Razorpay's own linked-account KYC mitigates much of this, but not the typo case. We recommend the client either enables penny-drop verification as a scope addition, or accepts a documented manual check of the cancelled cheque against the entered account number during admin review. This needs a decision before go-live.

7. Flow 4 — Automated vendor settlement

This is the "auto payout" for vendors. It runs without human intervention on a per-sub-order basis, and it works by transferring the vendor's share *early but held*, then releasing it when the order is safely delivered.

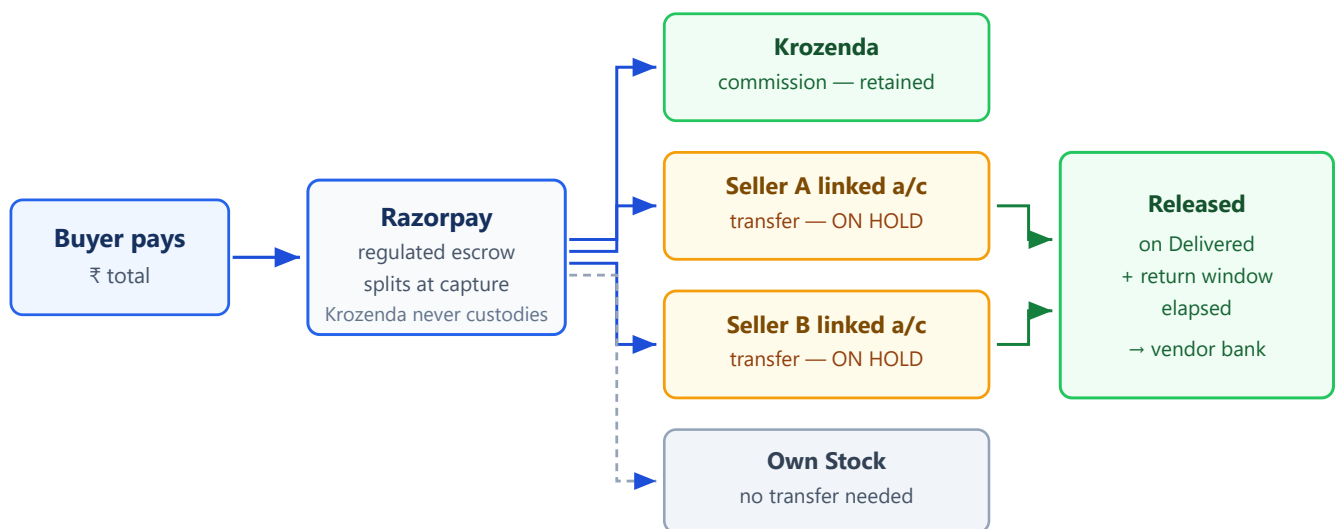


Figure 3 — Route settlement. The vendor's share is ring-fenced at capture and released only when the order is safe.

7.1 Why hold, rather than transfer late

There are two ways to run Route: create the transfer immediately but hold it, or wait until delivery and create it then. We recommend **transfer-on-hold at capture**, for three reasons:

- **The money is ring-fenced from day one.** The vendor's share is allocated against that specific payment, so a later dispute cannot leave it unfunded.
- **Reversal is clean.** A cancellation or refund reverses a known transfer against a known payment, instead of requiring us to decide whether to create one at all.
- **The vendor can see it.** "₹4,200 allocated, releasing on 14 March" is a far better Seller Panel experience than silence until settlement day, and it materially reduces settlement support tickets.

7.2 The release gate

A held transfer is released automatically when **all** of the following are true:

Condition	Detail
Status is Delivered	Confirmed by the logistics partner's delivery webhook, not by the vendor's own claim.
Return window elapsed	[48] hours from delivery, matching the return-request window in the Terms & Conditions.
No open claim	No return, replacement or damage claim is pending on that sub-order.
No RTO in flight	The shipment has not been marked for return to origin.
Vendor account in good standing	Not suspended, and no active fraud investigation or regulatory hold.

A scheduled job evaluates eligible sub-orders on the vendor's configured cycle — automatic, weekly or monthly. Where the cycle is *automatic*, release happens as soon as the gate opens. Where it is weekly or monthly, eligible transfers accumulate and release together on the cycle date, which suits vendors who prefer one consolidated credit and one reconciliation.

7.3 Settlement record

Every release writes an immutable settlement record carrying the full arithmetic, so that a vendor query years later can be answered from data rather than from reconstruction:

```
gross sale value
- platform commission (snapshotted at order time)
- payment gateway charge, where borne by the vendor
- shipping charge, where borne by the vendor
- penalties, RTO costs, and prior-cycle recoveries
- tax deducted at source
= net settled → transfer id · UTR · status · timestamp
```

8. Worked example with real numbers

A retail customer checks out a cart containing one item from Seller A and one from Seller B. Figures below use a [10%] commission on A and [8%] on B, and are illustrative.

Line	Seller A	Seller B	Total
Item value (incl. GST)	₹ 6,000	₹ 4,000	₹ 10,000
Shipping charged to buyer	₹ 100	₹ 100	₹ 200
Buyer pays	—	—	₹ 10,200
Commission rate (snapshotted)	10%	8%	—
Krozenda commission	₹ 600	₹ 320	₹ 920
Shipping borne by seller	₹ 100	₹ 100	₹ 200
TCS under GST s.52 [1%]	₹ 60	₹ 40	₹ 100

Line	Seller A	Seller B	Total
TDS under IT s.194-O [1%]	₹ 60	₹ 40	₹ 100
Net to seller (held, then released)	₹ 5,180	₹ 3,500	₹ 8,680
Retained by Krozenda	—	—	₹ 1,320

Of the ₹10,200 collected, ₹8,680 is routed to the two sellers' linked accounts and ₹1,320 is retained by Krozenda — of which ₹920 is commission revenue, ₹200 is recovered shipping, and ₹200 is tax collected and deducted that Krozenda must deposit with the authorities, not keep. Gateway fees are charged by Razorpay against the platform account on its own schedule and are not shown in this split.

Confirm before build. The TCS and TDS rates and — more importantly — whether Krozenda or the seller is the deductor for each business model, depend on the seller-of-record decision still open in section 19. The arithmetic above assumes Krozenda deducts. If that changes, the split changes.

9. Flow 5 — Refunds and partial refunds

A refund is not a payout. It reverses the original payment back to the instrument the customer paid with — the same card, the same UPI handle — through the gateway's Refunds API. Money never leaves Krozenda's bank account to make a refund happen.

9.1 The two halves of a refund

Every refund on a multi-vendor order has two movements that must both succeed:

Movement	What it does
Refund to buyer	Returns the sub-order's value to the buyer's original payment instrument.
Transfer reversal	Claws back the corresponding held or released transfer from the vendor's linked account, so the vendor is not paid for a sale that was undone.

These must be treated as one atomic business operation. Refunding the buyer without reversing the transfer means Krozenda has funded the refund out of its own money — the single most common and most expensive bug in marketplace settlement. The implementation records both legs against the sub-order and reconciles them nightly; any sub-order with one leg complete and the other not is raised as an exception the next morning.

9.2 Timing determines the mechanism

When	What happens
Before dispatch — buyer cancels	Transfer is still on hold. It is reversed directly and the buyer is refunded in full including shipping. Cleanest case.
After delivery, claim upheld	If the transfer is still held, reverse and refund. If already released, a reversal is raised against the vendor's linked account; where its balance is insufficient, the amount is recovered from the vendor's next settlement cycle.

When	What happens
Partial — one item of several	Only that sub-order's proportional value is refunded and only its transfer reversed. Sibling sub-orders are untouched.
Outside the gateway's refund window	The original payment can no longer be reversed. Compensation is paid instead by RazorpayX Payout to the customer's bank account — one of the few legitimate customer-facing uses of Rail B. Requires finance approval.

What the buyer experiences. Refunds are initiated within **[24 to 48]** hours of approval, but the credit appears in **[5 to 7]** working days because the last leg belongs to the buyer's bank, not to us. Setting that expectation in the app at the moment the refund is approved — with the reference number visible — is the difference between a satisfied customer and a support ticket on day three.

10. Flow 6 — RTO and settlement reversal

A Return to Origin occurs when a shipment cannot be delivered — a wrong address, a customer unreachable after repeated attempts, or a refused delivery. RTO is not covered by the No-Return policy and has its own money path.

RTO Initiated → shipment in return transit → RTO Delivered (back to vendor)

↓

transfer reversed • inventory restored • buyer refunded per allocation rule

Cause of the RTO	Money outcome
Courier failure or vendor error	Buyer refunded in full including shipping. Vendor's transfer reversed. Return freight borne per the configured platform/vendor rule.
Buyer-attributable — bad address, unreachable, refused	Buyer refunded the product value; forward and return freight actually incurred may be deducted and is disclosed before deduction. Vendor's transfer reversed.

In both cases inventory is restored for Own Stock and Marketplace models, and the reversal is written to the settlement ledger against the original transfer id so the vendor's statement reconciles.

11. Automated payroll via RazorpayX Payouts

Everything to this point has been marketplace money on Rail A. Payroll is different: it is Krozenda paying its own staff out of its own funds, and it belongs on Rail B.

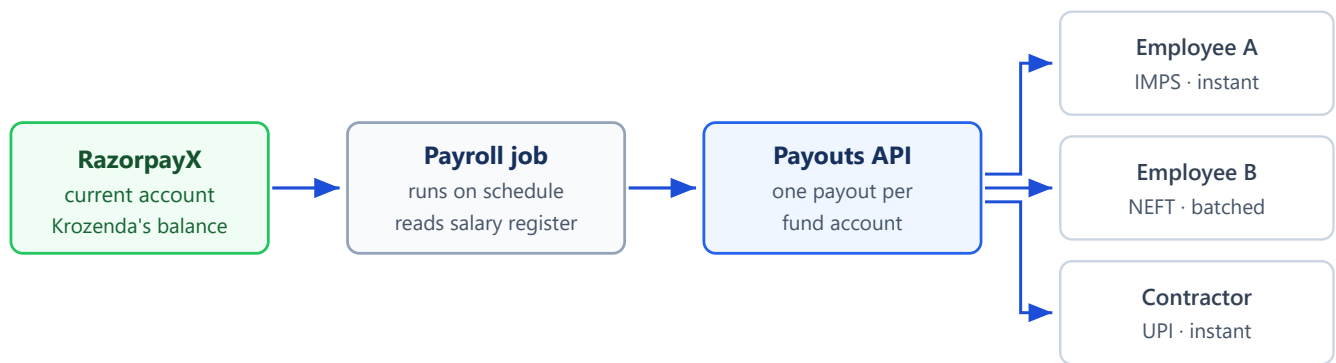


Figure 4 — Automated payroll. Krozenda's own balance, disbursed on a schedule to verified fund accounts.

11.1 What "automated payroll" means here

The platform holds a salary register — employee, designation, monthly gross, deductions, net payable, and a verified bank account or UPI handle. On the configured pay date a scheduled job reads the register, generates one payout instruction per employee, submits them, and tracks each to completion. Finance approves the run; the system executes it.

Capability	Behaviour
Scheduled run	Fires on <u>[the last working day of each month]</u> , or on demand for off-cycle payments.
Maker–checker approval	The run is prepared by one user and released by another. No single person can move payroll money alone.
Balance pre-check	Total run value is compared against available balance before submission; a short balance halts the run rather than paying some staff and not others.
Per-employee tracking	Each payout carries its own status and UTR, visible in the admin panel.
Automatic retry	A payout that fails for a transient reason is retried on the next cycle; one that fails for a bad account is flagged for correction and never retried blindly.
Audit trail	Who prepared, who approved, what was sent, what was returned — retained for statutory audit.

Scope note. This is *salary disbursement*, not a full payroll system. Computing gross-to-net — PF, ESI, professional tax, TDS on salary under section 192, leave and attendance, Form 16 — is a separate body of work and is not in the current scope. What is described here takes an approved net-payable figure and moves the money reliably. If the client needs statutory payroll computation, that should be scoped separately or handled in dedicated payroll software that hands us the net figures.

12. The Payout API in detail

RazorpayX models a payout in three layers. Each layer is created once and reused, which keeps the recurring monthly run cheap and safe.

Layer	Created	Holds
Contact	Once per person or entity	Name, email, phone, type — employee, vendor, customer. The <i>who</i> .
Fund account	Once per bank account or UPI handle	Account number and IFSC, or the VPA. Belongs to a contact. The <i>where</i> .
Payout	Once per payment	Amount, mode, purpose, reference, narration. The <i>what and when</i> .

```
// Monthly salary run – one instruction per employee
POST /v1/payouts
X-Payout-Idempotency: "payroll-2026-03-emp-00417" // prevents double payment on retry
{
  "account_number": "[Krozenda RazorpayX account]",
  "fund_account_id": "fa_XXXXXXXXXXXX",
  "amount": 4250000, // paise – ₹42,500.00
  "currency": "INR",
  "mode": "IMPS", // IMPS | NEFT | RTGS | UPI
  "purpose": "salary",
  "queue_if_low_balance": true, // hold, don't fail, if balance is short
  "reference_id": "PAYROLL-2026-03-00417",
  "narration": "Krozenda Salary Mar 2026"
}
```

12.1 Choosing the mode

Mode	Speed	Use it for
IMPS	Immediate, 24×7	Salaries and anything time-sensitive. Per-transaction ceiling applies.
NEFT	Batched, banking hours	Bulk non-urgent transfers where cost matters more than speed.
RTGS	Real-time, banking hours	High-value single transfers above the RTGS floor.
UPI	Immediate, 24×7	Small-value payouts to a VPA. Lower ceiling than IMPS.

12.2 Payout lifecycle

```

queued → pending → processing → processed      ✓ money delivered, UTR issued
                                     ↘ reversed      X bank returned it – funds back in your balance
                                     ↘ failed          X never left – safe to correct and retry
    ↘ rejected      X validation or approval refused it
    ↘ cancelled      – cancelled while still queued

```

The distinction between **failed** and **reversed** matters operationally. *Failed* means the money never left — correct the account details and retry. *Reversed* means it left, the beneficiary bank rejected it, and it came back; the employee has not been paid, the balance has been restored, and finance must be told. Both must be surfaced distinctly in the admin panel rather than collapsed into "error".

Idempotency is not optional. Every payout request must carry a unique idempotency key derived from the payroll period and the employee — never a random value, or the protection is lost. Without it, a network timeout on a request that actually succeeded will, on retry, pay the employee twice. Recovering a duplicate salary payment is a manual, relationship-damaging process. This is the single highest-risk detail in the payout implementation.

13. Payout automation and scheduling

The automated run executes in this order, and stops at the first failure rather than proceeding partially:

1. **Assemble.** Read the salary register for the period; exclude leavers and unapproved rows.
2. **Validate.** Confirm every included employee has an active fund account and a non-zero net payable.
3. **Total and check balance.** Compare the run value against the available RazorpayX balance.
4. **Present for approval.** Finance sees headcount, total value and exceptions, and approves or rejects the whole run.
5. **Submit.** One payout per employee, each with its own idempotency key, submitted with controlled concurrency to stay inside rate limits.
6. **Track.** Persist every payout id against its register row; update status from webhooks.
7. **Reconcile and report.** Once all payouts are terminal, produce the run report: paid, failed, reversed, pending — with UTRs.

The same machinery serves off-cycle disbursements — a vendor incentive, a customer goodwill payment outside the refund window, an employee reimbursement — differing only in the `purpose` field and the approval threshold.

14. Webhooks and reconciliation

Webhooks are how the platform learns what actually happened. Every one is verified by HMAC signature against the webhook secret before it is trusted, and rejected outright if the signature does not match.

Event	What the platform does
<code>payment.captured</code>	Confirm the order, create sub-orders, notify vendors.
<code>payment.failed</code>	Mark the attempt failed; leave the cart intact so the buyer can retry.
<code>order.paid</code>	Cross-check against our own order total; flag any mismatch.
<code>refund.processed</code>	Close the refund, notify the buyer, complete the ledger entry.
<code>refund.failed</code>	Raise an exception for finance — the buyer is still owed money.
<code>transfer.processed</code>	Mark the vendor's settlement complete and publish it to the Seller Panel.
<code>transfer.failed</code>	Hold the settlement, alert admin, block further releases to that vendor pending investigation.
<code>payout.processed</code>	Mark the salary or disbursement paid; record the UTR.
<code>payout.reversed</code>	Mark unpaid, restore the amount to the run's outstanding total, alert finance.

Event	What the platform does
<code>payout.failed</code>	Flag for correction; do not retry automatically without a human check.

14.1 Three rules the implementation follows

- **Verify before trusting.** An unsigned or badly-signed webhook is discarded and logged as a security event.
- **Handle idempotently.** Razorpay may deliver the same event more than once. Every handler is safe to run repeatedly — a second `payment.captured` for an order already confirmed changes nothing.
- **Never assume delivery.** Webhooks can be missed entirely. A nightly reconciliation job independently fetches the day's payments, refunds, transfers and payouts from Razorpay and compares them against our records, correcting drift and reporting anything it cannot reconcile.

Why the nightly job earns its keep. Webhook delivery is reliable but not guaranteed — an outage on our side during a delivery window loses events permanently. Without independent reconciliation, a missed `transfer.processed` means a vendor is paid but the Seller Panel says otherwise, and the discrepancy is found by the vendor rather than by us. Reconciliation converts an unbounded class of silent errors into a bounded morning report.

15. Failure handling and edge cases

Scenario	Handling
Buyer debited, order not created	The nightly reconciliation finds the orphan payment and auto-refunds it. The buyer is notified with the reference.
Duplicate payment for one cart	Detected on the order reference; the second payment is refunded automatically.
Vendor's linked account suspended mid-order	The transfer stays on hold. The order still ships and the buyer is unaffected. Settlement releases once the account is restored, or is reversed if the vendor is terminated.
Transfer reversal exceeds the vendor's balance	The shortfall becomes a recoverable owed to the platform, deducted from the next cycle and carried forward until cleared.
Refund requested after gateway window	Escalates to a manual Payout with finance approval, as described in section 9.2.
Payroll run interrupted mid-submission	Idempotency keys make resubmission safe. The run resumes and pays only those not already paid.
RazorpayX balance short on pay date	<code>queue_if_low_balance</code> holds the payouts rather than failing them; they execute automatically once funded. Finance is alerted immediately.
Razorpay outage during checkout	Checkout degrades gracefully with a clear message. No order is created without a verified payment, so nothing is left half-done.
Delivery webhook never arrives	Settlement stays gated. A stale-order report lists sub-orders shipped but not marked delivered beyond a threshold, for manual review — deliberately failing closed rather than paying out on an unconfirmed delivery.

16. Security and key management

- **No card data on our servers.** Card numbers, CVV, UPI PINs and net-banking credentials are captured on Razorpay's PCI-DSS compliant interface. Krozenda stores only the payment reference, amount and status.
- **Amounts are server-authoritative.** Every payable amount is computed on the server from the database. A price or total submitted by the client is never trusted.
- **Signature verification everywhere.** Both the checkout callback and every webhook are HMAC-verified before any state changes.
- **Key segregation.** Payment gateway keys and RazorpayX payout keys are separate credentials with separate blast radii, held in environment configuration, never in source control, and rotated on a schedule. Test and live keys are never present in the same environment.
- **Payout authorisation is privileged.** Initiating or approving a payout is a distinct permission held by a small number of finance roles, separate from general admin, and every action is written to the immutable audit trail.
- **Maker-checker on money out.** No single account can both prepare and release a payout run.
- **Rate limiting and replay protection** on all payment endpoints, with webhook timestamps checked to reject replayed deliveries.

17. Tax treatment — TCS and TDS

As a marketplace operator, Krozenda has collection and deduction obligations on vendor sales:

Obligation	Treatment
TCS — GST s.52	Collected on the net taxable value of vendor supplies, deducted before settlement, deposited and reported in GSTR-8. The vendor claims credit in its own returns.
TDS — IT s.194-O	Deducted on the gross sale value of vendor supplies, deposited, and reflected in the vendor's Form 26AS.
Own Stock sales	No TCS or TDS — Krozenda is the seller, not the facilitator. Ordinary GST output liability applies.
Salary payouts	TDS under s.192 is a payroll computation, outside this scope. Payouts move the approved net figure.

Both deductions appear as itemised lines on every settlement statement, and the platform generates period-wise reports for filing.

18. What the client must provide

Item	Owner	Blocking?
Razorpay account with Route provisioned	Client	Yes — no vendor settlement can be built or tested without it
RazorpayX current account, activated for Payouts	Client	Yes — for payroll

Item	Owner	Blocking?
Live and test API keys for both products	Client	Yes
Webhook secrets configured on both	Client	Yes
Commission structure — rates by product, vendor, category	Client	Yes
Settlement cycle policy per vendor tier	Client	Yes
Return window length, feeding the release gate	Client	Yes
Salary register format and the approval hierarchy	Client	For payroll
Decision on penny-drop verification (section 6)	Client	Before go-live
Seller-of-record and TCS/TDS position	Client	Yes — drives invoicing and the split

19. Open decisions

#	Decision	Why it must be settled before build
1	Is Razorpay Route provisioned?	Route is not enabled by default and requires approval from Razorpay. Nothing in vendor settlement can be built against it until it exists on the client's account. This is the critical path item.
2	Seller of record per business model	Determines who invoices the customer, who carries consumer-law liability, and who deducts TCS and TDS. It changes the settlement arithmetic in section 8 and the Terms & Conditions.
3	Transfer on hold, or transfer on delivery?	We recommend on-hold at capture (section 7.1). The client should confirm, as it affects what vendors see and when.
4	Penny-drop verification in or out?	Currently out of scope. Without it, bank-detail typos surface as failed transfers rather than being caught at entry.
5	Who bears the gateway fee?	Platform-absorbed or deducted from the vendor. Materially affects vendor economics and must be stated in the Vendor Agreement.
6	Payroll scope	Confirm that disbursement-only is intended, and that statutory payroll computation is handled elsewhere (section 11.1).

20. Implementation sequence

Each stage is independently testable and delivers something demonstrable:

Stage	Scope	Depends on
1	Payment capture — order creation, checkout, signature	Gateway keys

Stage	Scope	Depends on
	verification, webhooks, order confirmation	
2	Order split, commission resolution and snapshot onto sub-orders	Commission structure
3	Linked account onboarding, KYC state machine, storefront gating	Route provisioned
4	Transfers on hold, release gate, settlement scheduler, settlement statements	Stage 3, return window policy
5	Refunds, partial refunds, transfer reversal, the atomic two-leg guarantee	Stage 4
6	RTO reversal and inventory restoration	Stage 5, logistics webhooks
7	Nightly reconciliation and the exception report	Stages 1–6
8	Payouts — contacts, fund accounts, payroll run, maker–checker, payout webhooks	RazorpayX account
9	TCS/TDS deduction, statements and filing reports	Decision 2

Stages 1 and 2 can begin immediately. Stage 3 onwards is blocked on Route provisioning, and stage 8 on the RazorpayX account — which is why both appear as blocking items in section 18.

Summary of the compliance position. Vendor sale proceeds move on Razorpay Route and are never custodied by Krozenda. Krozenda's own funds — commission, own-stock revenue — settle to Krozenda's account and are disbursed through RazorpayX Payouts for salaries and platform expenses. Keeping these two rails separate is what allows Krozenda to operate a multi-vendor marketplace without holding a Payment Aggregator licence.

Krozenda Payment & Settlement Flow · Version 1.0 · [DD Month YYYY] · Prepared by Appzeto for [Legal Entity Name Pvt. Ltd.]